DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 1:
Introduction and Overview
Memory Hierarchies and Matrix-Vector Multiplication

DS 642: Applications of Parallel Computing

Course Main Website

Course on GitHub

Computing Systems

Throughout the course, I use a lot of resources from:

– UC Berkeley CS267

– Cornell CS 5220

Objectives

- Learn about applications of high-performance and parallel computing
- Hands-on experience of parallel platforms
- Develop and tune codes based on existing codes
- Measure and test for performance
- Apply software development practices

Lecture Plan

- Intro to parallel computers
- Parallel computer architectures
- Parallel performance
- Motifs of parallel computing

Parallel Computers

- It's all about the need for speed.
- Powerful computers are parallel computers.
- Large computational science and engineering problems require powerful computers.

Examples:

Climate modeling

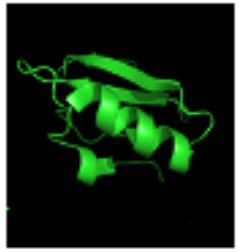
Drug discovery

Energy harvesting

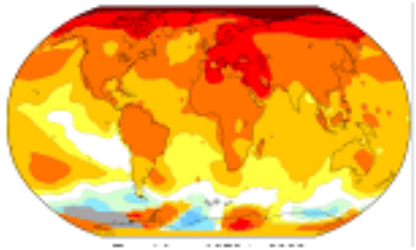
High performance computing is used to study things that are:

- too big
- too small
- too fast
- too slow
- too expensive or
- too dangerous

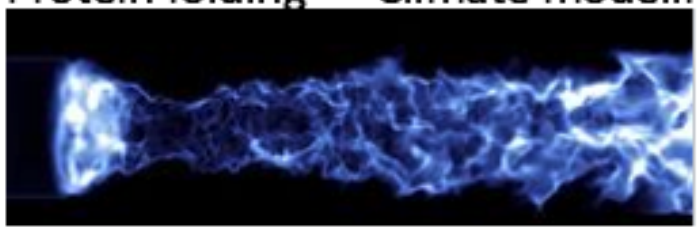
for experiments.



Protein folding



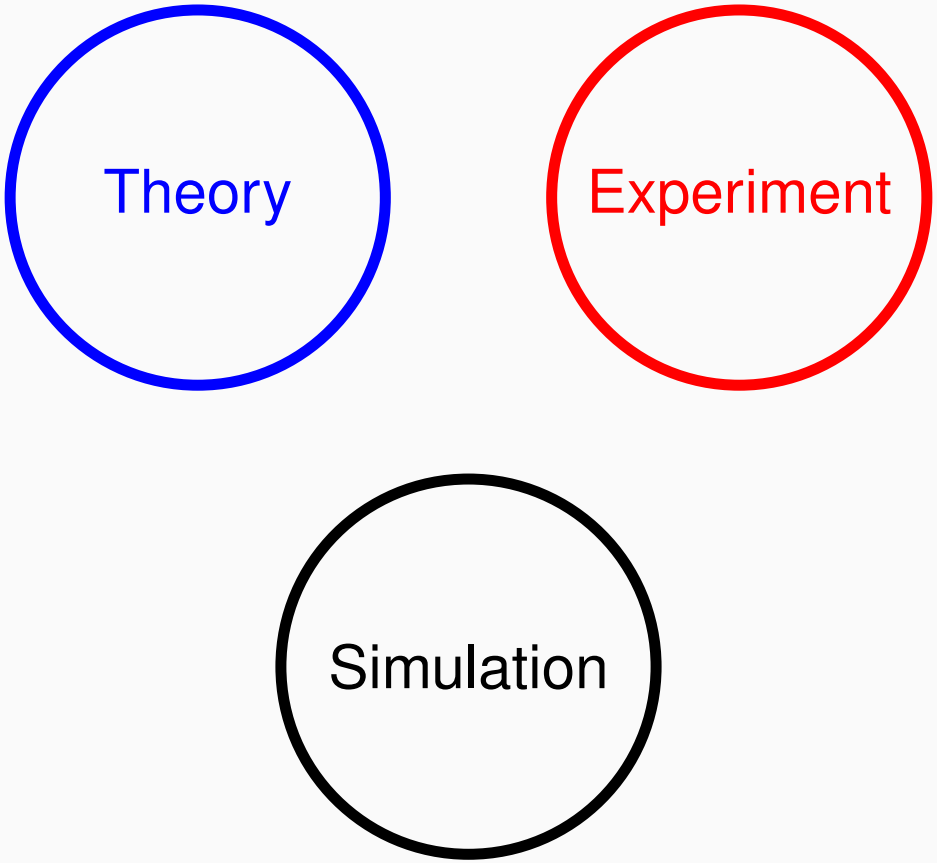
Climate modeling



Combustion

Simulation in Science and Engineering

Simulation: The Third Pillar of Science



Parallel Computers

- Data analysis demands more computing power.
- Scientific data sets are growing exponentially.
- Ability to generate data is exceeding our ability to store and analyze.

Examples:

Gnome

Sensor data

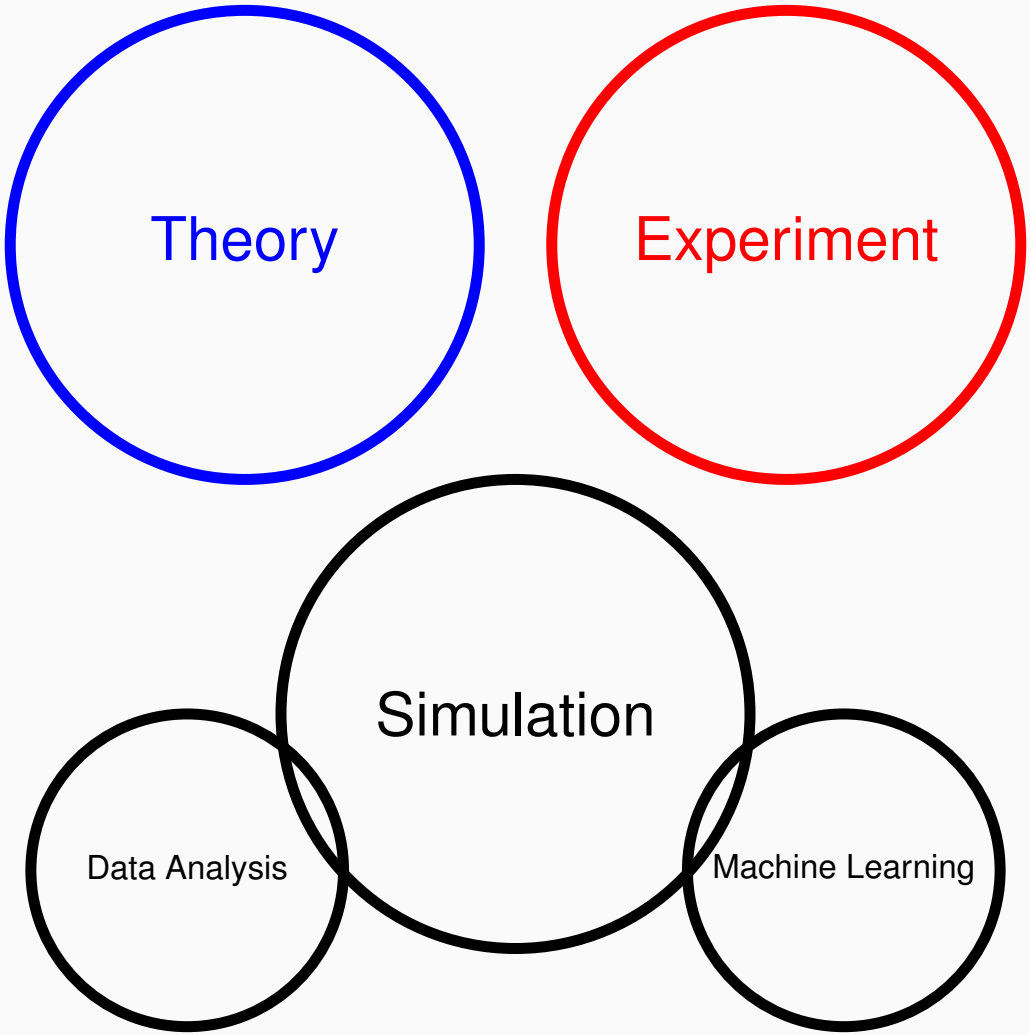
Telescope solar images

High Performance Data Analytics is used to study data sets that are:

- too big
- too complex
- too fast (streaming)
- too noisy
- too heterogeneous

Data growth is outpacing computing growth

Machine learning demands more computing



From 2011-2017 the petaflop/sec-day training has increased by a factor of 300,000.

Why Parallel Computers?

- We need ever-increasing performance.
- Instead of designing and building faster microprocessors, put multiple processors on a single integrated circuit.
- Writing parallel programs aren't easy and serial programs don't benefit from multi-processor systems.
- Running multiple instances of a serial program (concurrency) isn't often very useful.

Motifs: Common applications of computational methods

- Dense/Sparse Linear Algebra
- Monte Carlo
- Mesh generation, graph partitioning and load balance
- Fast Methods (e.g. FFT)

Why parallel computing?

Need for speed and memory

- Many processors together in parallel solve problems more quickly.
- Parallel computers may use distributed memory model (multiple processors with their own memory).
- Concurrently running multiple tasks that are logically active at one time is NOT parallel computing vs parallelism where multiple tasks are actually active at one time.

The fastest computers have gone parallel for a long time now.

Units of Measure for HPC

High Performance Computing (HPC) units are:

- Flop: floating point operation, usually double precision unless noted
- Flop/s: floating point operations per second
- Bytes: size of data (a double precision floating point number is 8 bytes)

Speed measured in flop/s (floating point ops / second):

- Giga (10^9 flop/sec) – a single core
- Tera (10^{12} flop/sec) – a big machine
- Peta (10^{15} flop/sec) – current top 10 machines (5 in US)
- Exa (10^{18} flop/sec) – goal

Current fastest (public) machines are petaflop systems:

http://www.top500.org

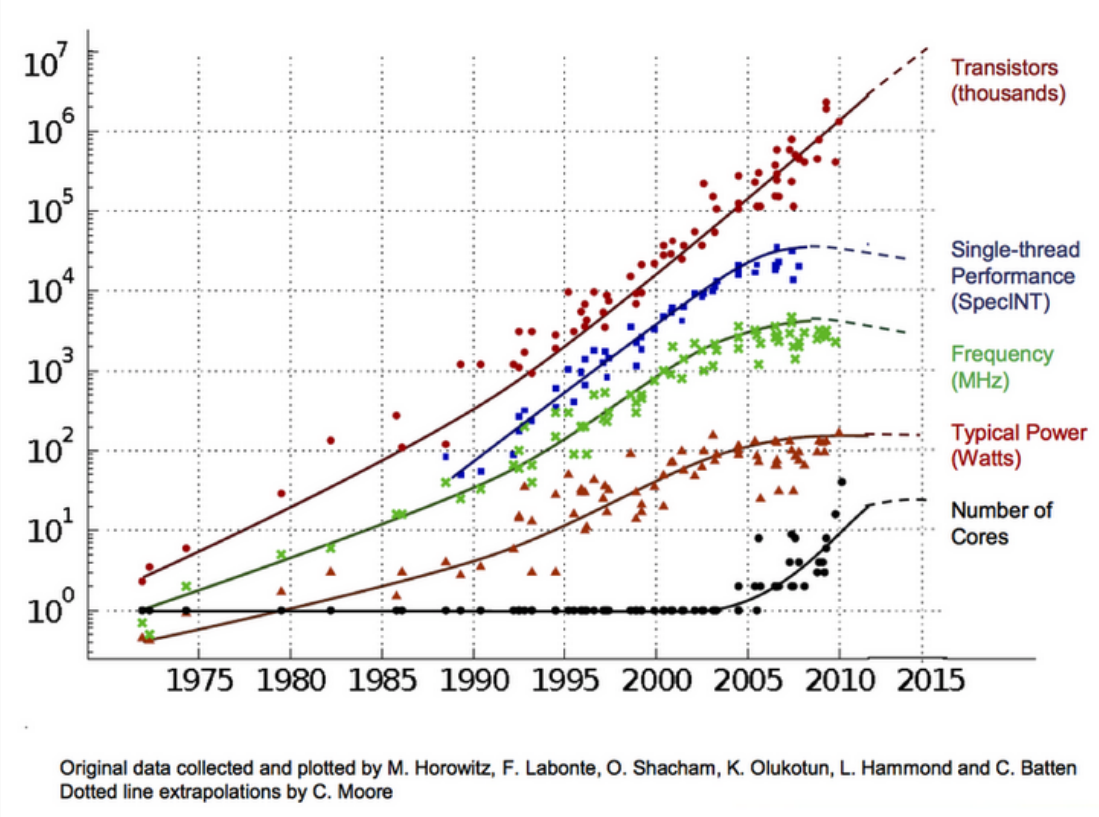
Current fastest (public) machines, as of November 2024

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	
5	**HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, HPE Eni S.p.A. Italy	3,143,520	477.90	606.97	8,461

http://www.top500.org

Why parallel computing?

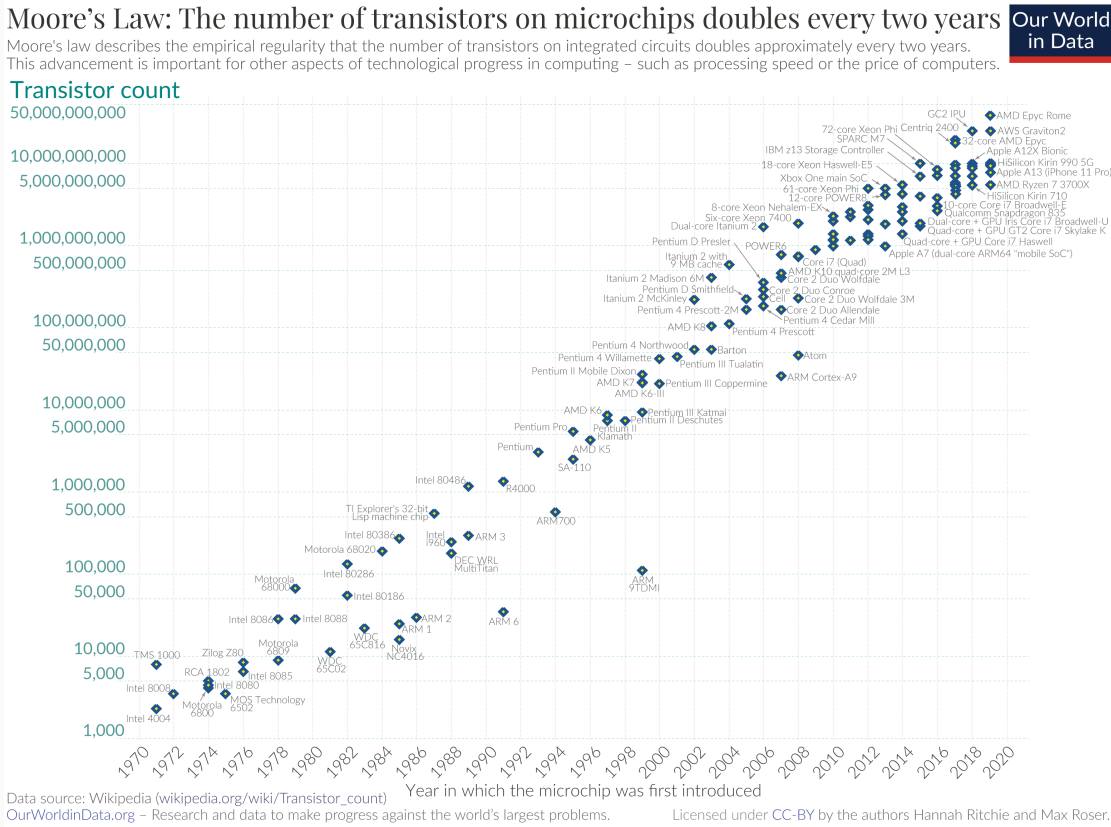
- Gordon Moore (co-founder of Intel) predicted in 1965 that the transistor density of semiconductor chips would double roughly every 18 months.
- Microprocessors have become smaller, denser, and more powerful.
- There's an inherent problem in device shrinkage - power density limits serial performance.



Since 2000, microprocessors performance has not increased as predicted.

Why parallel computing? Moore's law reinterpreted

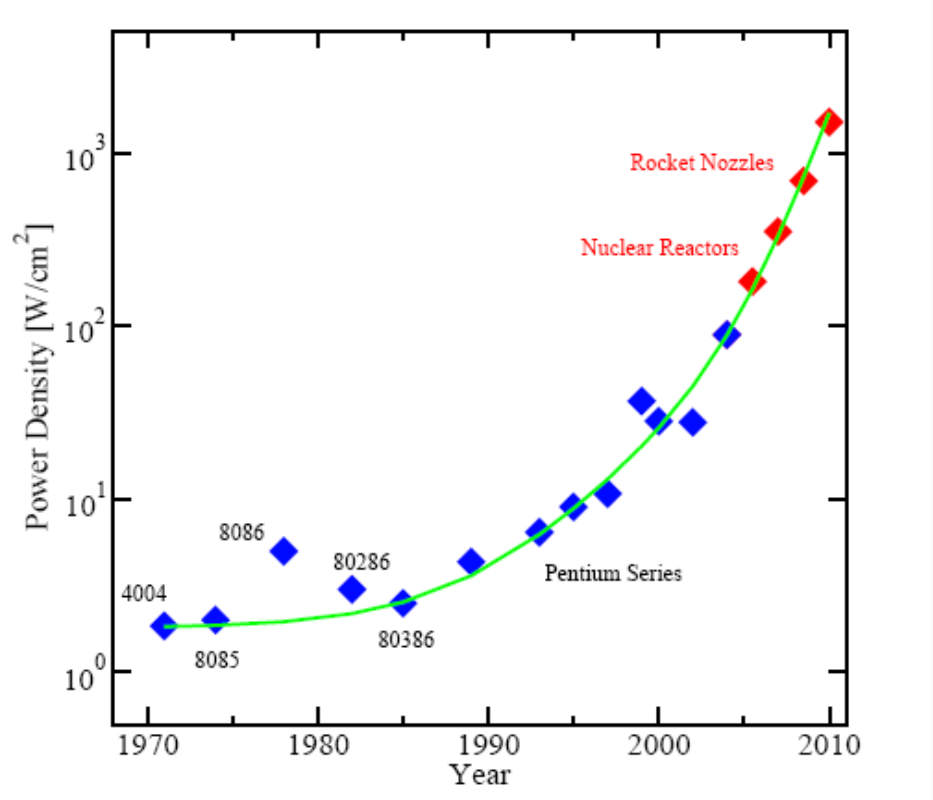
- Number of cores per chip can double every two years.
- Clock speed will not increase.
- Need to deal with parallel systems:
  - Need to deal with inter-chip parallelism.
  - Need to deal with intera-chip parallelism.
- The actual Moore's law seems alive, for now.



Plot of transistor counts for microprocessors against dates of introduction.

Why parallel computing?

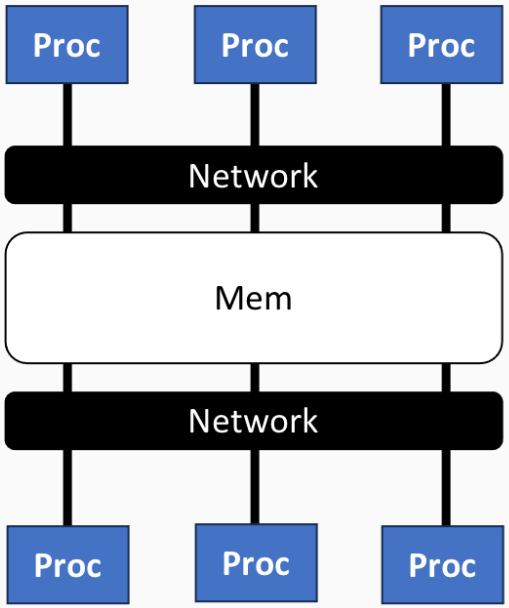
- Smaller transistors lead to increased power consumption.
- Increased heat is caused by increased power consumption.
- High performance serial processors waste power; the solution is more transistors, but not faster serial processors.



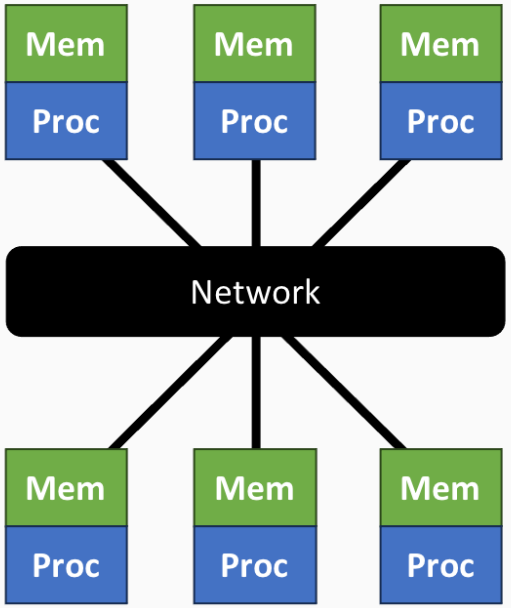
Evolution of selected microprocessors power density.

Source: Patrick Gelsinger, Shenkar Bokar, Intel.

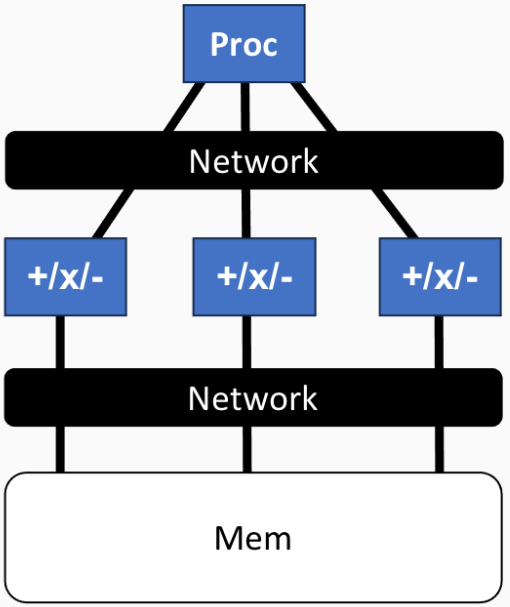
Types of Parallel Systems



Shared Memory (SMP) or Multicore



High Performance Computing (HPC) or Distributed Memory



Single Instruction Multiple Data (SIMD)

Examples

Embarrassingly parallel problem: Compute the prime factors of 1 billion numbers: divide the work between 1 million processors.

Other examples:

- Monte Carlo computations with many independent trials
- Big data computations mapping many data items independently

Examples

Problem with dependencies: Add n numbers - in serial, it takes $O(n)$ time to compute.

Serial code:

```
1  sum = 0;
2  for (i = 0; i < n; i++) {
3    x = fetch_next_value (. . .);
4    sum += x;
5  }
```

Examples

Parallel code with P processors:

```
1  my_sum = 0;
2  my_first_i = . . . ;
3  my_last_i = . . . ;
4  for (my_i = my_first_i; my_i < my_last_i; my_i++) {
5    my_x = fetch_next_value (. . .);
6    my_sum += my_x;
7  }
```

- Each core completes execution of this code
- When cores are done computing their values of `my_sum`, they can form a global sum by sending their results to a designated “master” core to add their results:

Examples

Parallel code with P processors:

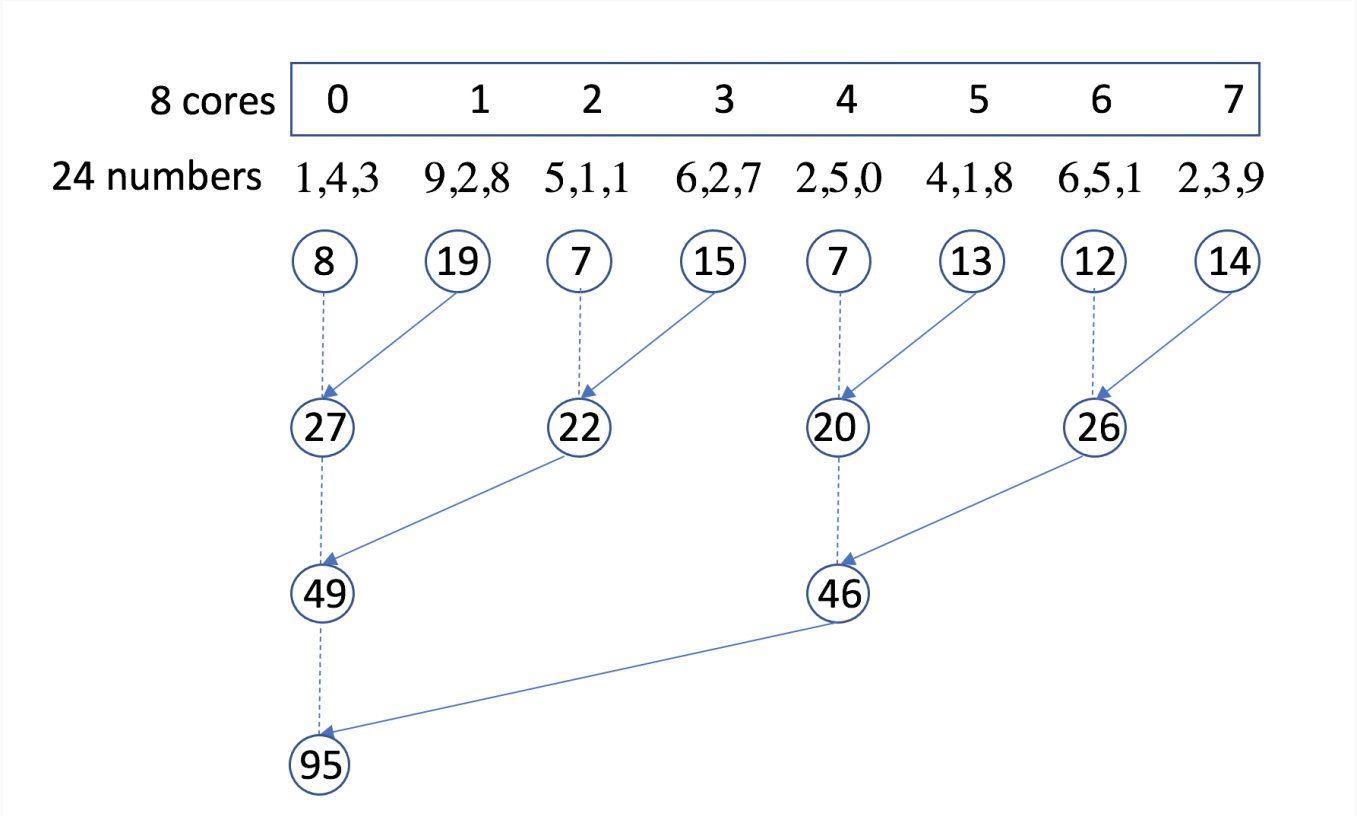
```
1  if (I'm the master core) {
2    sum = my_x;
3    for each core other than myself {
4      receive value from core;
5      sum += value;
6    }
7  }
8  else {
9    send my_x to the master;
10 }
```

Compute `my_sum` in parallel, but finish before adding all the partial sums.

Can we do better?

Examples: using parallelism to solve problems more quickly

Add up different pairs of numbers in parallel, and continue the process until we get a single answer - parallel time using n processors is $O(\log n)$.



Multiple processors forming a global sum.

Quantifying Parallel Performance

- Starting point: good *serial* performance
- Strong scaling: compare parallel to serial time on the same problem instance as a function of number of processors (p)

$$\text{Speedup} = \frac{\text{Serial time}}{\text{Parallel time}}$$

$$\text{Efficiency} = \frac{\text{Speedup}}{p} = \frac{1}{p}\frac{\text{Serial time}}{\text{Parallel time}}$$

- Ideally, speedup = p (linear speedup: Parallel time = Serial time/p). Usually, speedup $< p$.

Speedups and efficiencies of a parallel program

p	1	2	4	8
speedup	1	1.9	3.6	6.5
efficiency	1	0.95	0.9	0.81

- Barriers to perfect speedup
  - Serial work (Amdahl's law)
  - Parallel overheads (communication, synchronization)

Finding Enough Parallelism: Amdahl's Law

Parallel scaling study where some serial code remains:

$$p = \text{number of processors}$$

$$s = \text{fraction of work that is serial}$$

$$t_s = \text{serial time}$$

$$t_p = \text{parallel time} \geq st_s + (1-s)t_s/p$$

Amdahl's law:

$$\text{Speedup} = \frac{t_s}{t_p} \leq \frac{1}{s + (1-s)/p} \leq \frac{1}{s}$$

Even if the parallel part speeds up perfectly performance is limited by the sequential part.

So if 1% serial work, max speedup is less than $100\times$, regardless of p.

Principles of Parallel Computing

- Finding enough parallelism (Amdahl's Law)
- Granularity – how big should each parallel task be
- Locality — moving data costs more than arithmetic
- Load balance — don't want 1K processors to wait for one slow one
- Coordination and synchronization – sharing data safely
- Performance modeling/debugging/tuning

All of these things makes parallel programming even harder than sequential programming.

Objective:

Learning to write programs that are explicitly parallel:

- Serial programs don’t usually or fully benefit from multiple cores.
- Automatic parallel program generation from serial codes does not typically lead to getting high performance from parallel machines.
- Parallel programs involves communication between cores.
- Parallel programs are complex and difficult to develop, requiring good programming skills.

Overhead of Parallelism

Poor speed-up occurs because:

- The problem size n is small
- The communication cost is relatively large
- The serial computation cost is relatively large

Given enough parallel work, the biggest barrier to getting desired speedup is the parallelism overhead:

- cost of starting a thread or process
- cost of communicating shared data
- cost of synchronizing
- extra (redundant) computation

Overview of the course

Rough List of Topics:

1. **Basics:** Computer architecture, memory hierarchies, and performance.
2. **Algorithms/Machine Model:** Parallel programming models; parallel machines and architecture; parallel concepts, locality and parallelism in scientific codes.
3. **Parallel languages/libraries :** OpenMP, MPI, CUDA/OpenCL, cloud systems, compilers and tools.
4. **Motifs:** Monte Carlo, dense and sparse linear algebra and PDEs, graph partitioning and load balancing, fast transforms.

What you should get out of the course

In depth understanding of:

- When is parallel computing useful?
- Understanding of parallel computing hardware options.
- Overview of programming models (software) and tools, and experience using some of them
- Some important parallel applications and the algorithms
- Performance analysis and tuning
- Exposure to various open research questions

DS 642: Performance basics

Shahriar Afkhami

Week 1, part two

Performance basics

- Most applications run at $< 10\%$ of the “peak” performance.
- Much of the performance is lost on a single processor.
- Most of that loss is in the memory system (moving data takes much longer than arithmetic and logic).
- The road to good performance starts with a good serial code.
  - Even single-core performance is hard to optimize.
  - Optimized single core code is the building block of well-engineered libraries.

The Cost of Computing: Matrix Multiplication

Consider a simple serial code:

```
1   /* Matrix multiplication for n-by-n matrices */
2   for (i = 0; i < n; ++i)
3     for (j = 0; j < n; ++j)
4       for (k = 0; k < n; ++k)
5         C[i][j] = C[i][j] + A[i][k] * B[k][j];
6
```

Simplest model:

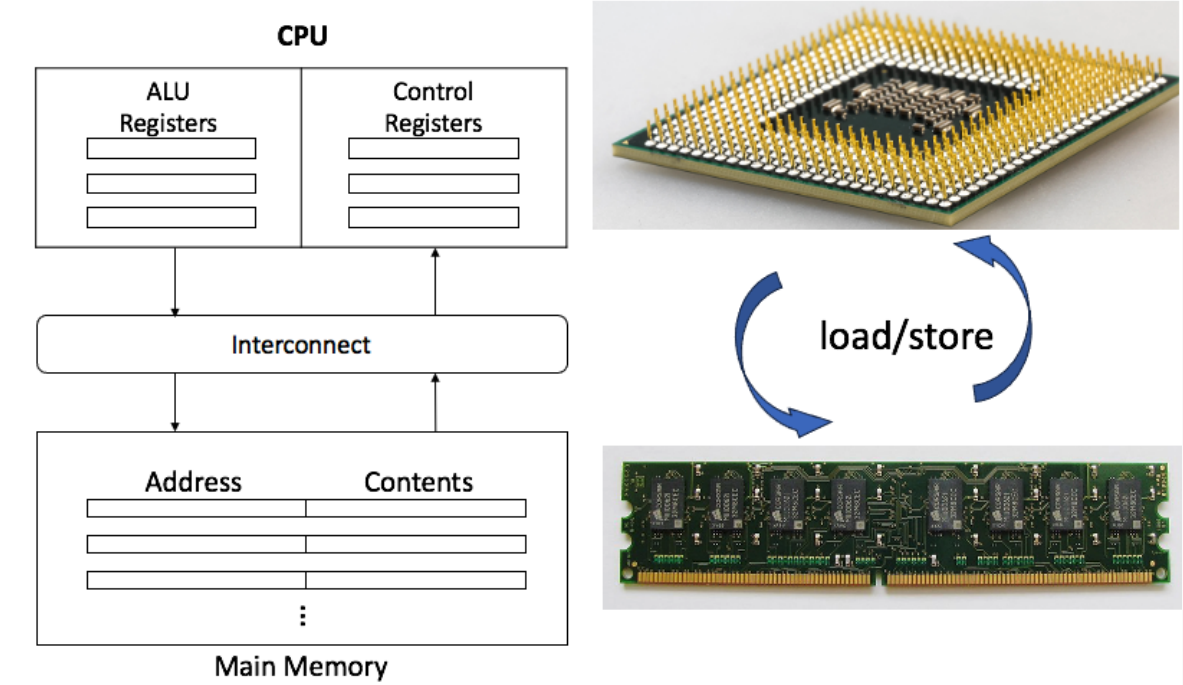
1. Dominant cost is $2n^3$ flops (adds and multiplies)
2. One flop per clock cycle
3. Expected time is

$$\text{Time (s)} \approx \frac{2n^3 \text{ flops}}{2.4 \cdot 10^9 \text{ cycle/s} \times 1 \text{ flop/cycle}}$$

Architectural organization

- Cores put together on central processing units (CPU's).
- Cores have instruction-level parallelism.
- There is a memory hierarchy.
- Accelerators (e.g. graphics processing units) possess similar architectural.
- CPU's and accelerators are put together in whats called nodes.
- Interconnect networks connect the pieces.

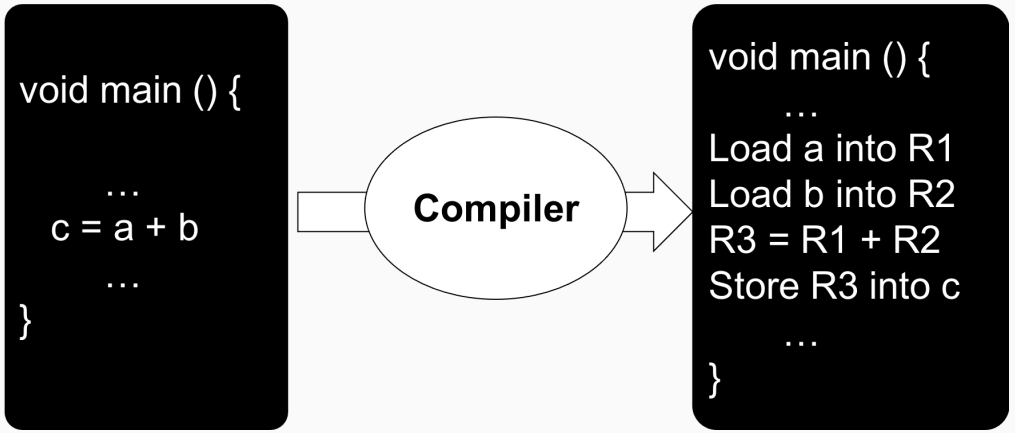
Central processing unit and main memory



- Main memory is where data and instructions are stored.
  - Every location consists of an address, which is used to access the location, and the contents of the location.
- Central processing unit (CPU) is divided into two parts:
  - Control unit
  - Arithmetic and logic unit (ALU)

Compilers and assembly code

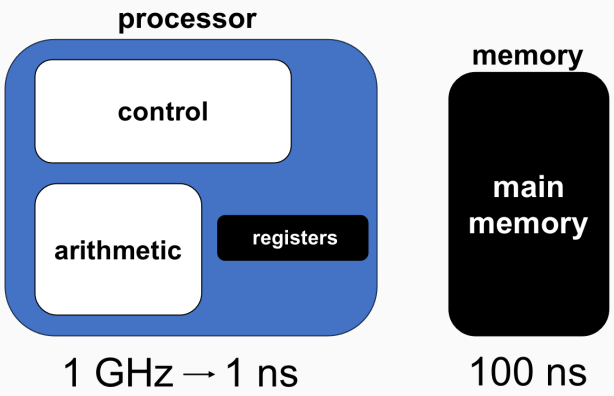
Compiler translates high level expressions into lower level instructions



- Compiler performs “register allocation” to decide when to load/store and when to reuse
  - Read address(a) to R1
  - Read address(b) to R2
  - R3 = R1 + R2
  - Write R3 to Address(c)
- Compiler Optimizes Code

Idealized Uniprocessor Model

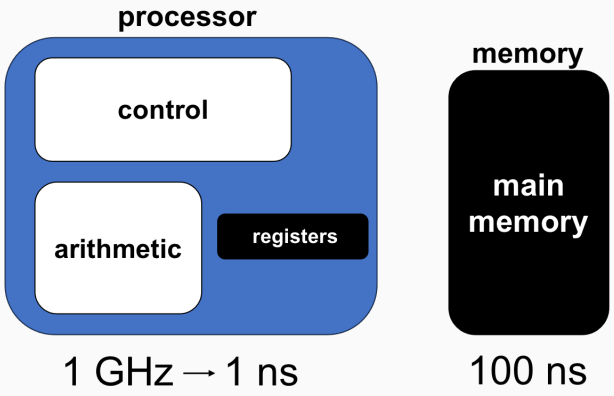
- Processor names variables, that are bytes, words, etc., in its address space.
  - Integers, floats, doubles, pointers, arrays, structures, etc.
  - These are really words, e.g., 64-bit doubles, 32-bit ints, bytes, etc.



- Processor performs operations on those variables.
  - Arithmetic, logical operations, etc.
  - Only performs these operations on values in registers

Idealized Uniprocessor Model

- Processor controls the order, as specified by program.
  - Branches (if), loops, function calls, etc.
  - Only performs these operations on values in registers
  - Load/Store variables between memory and registers
- Idealized Cost
  - Each operation (add, multiply, etc.) has roughly the same cost
  - Load/store is 100x the cost of add, multiply, etc.



Illustrating Idealized Uniprocessor Model

Latency vs Bandwidth

- Latency: the time that a CPU takes to get a block of data from the memory system.
- Bandwidth: the rate at which data can be pumped from the memory to the processor.

Illustrating Latency vs Bandwidth

Effect of memory latency on performance

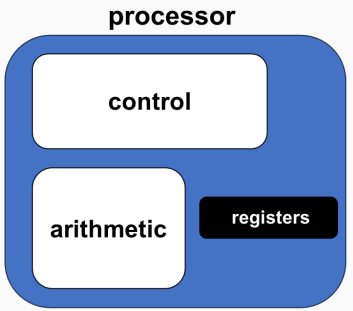
von Neumann Bottleneck: the transfer of data and instructions between memory and the CPU is inherently sequential.

Assume a CPU operates at 1GHz (1 ns clock) and is connected to a DRAM with a latency of 100 ns. Assume the CPU has 2 multiply-add units and is capable of executing 4 instructions in each cycle of 1 ns. The peak CPU rating is 4GFLOPS (floating-point operations per second).

Since the memory latency is 100 cycles, CPU must wait 100 cycles before it can process data. Therefore, the peak speed of computation is 10MFLOPS.

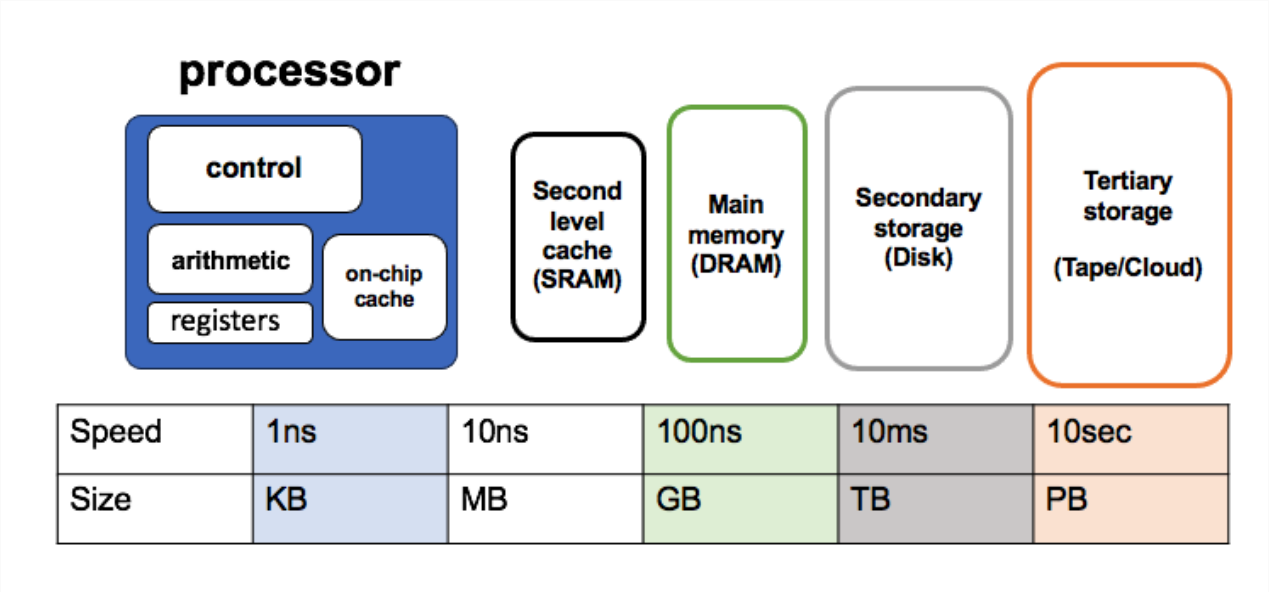
More Realistic Uniprocessor Model

- Memory accesses (load / store) have two costs
  - Latency: cost to load or store 1 word
  - Bandwidth: Average rate (bytes/sec) to load/store a large chunk



1 GHz → 1 ns

100 ns



Speed	1ns	10ns	100ns	10ms	10sec
Size	KB	MB	GB	TB	PB

Large memories are slow, fast memories are small

Cache basics

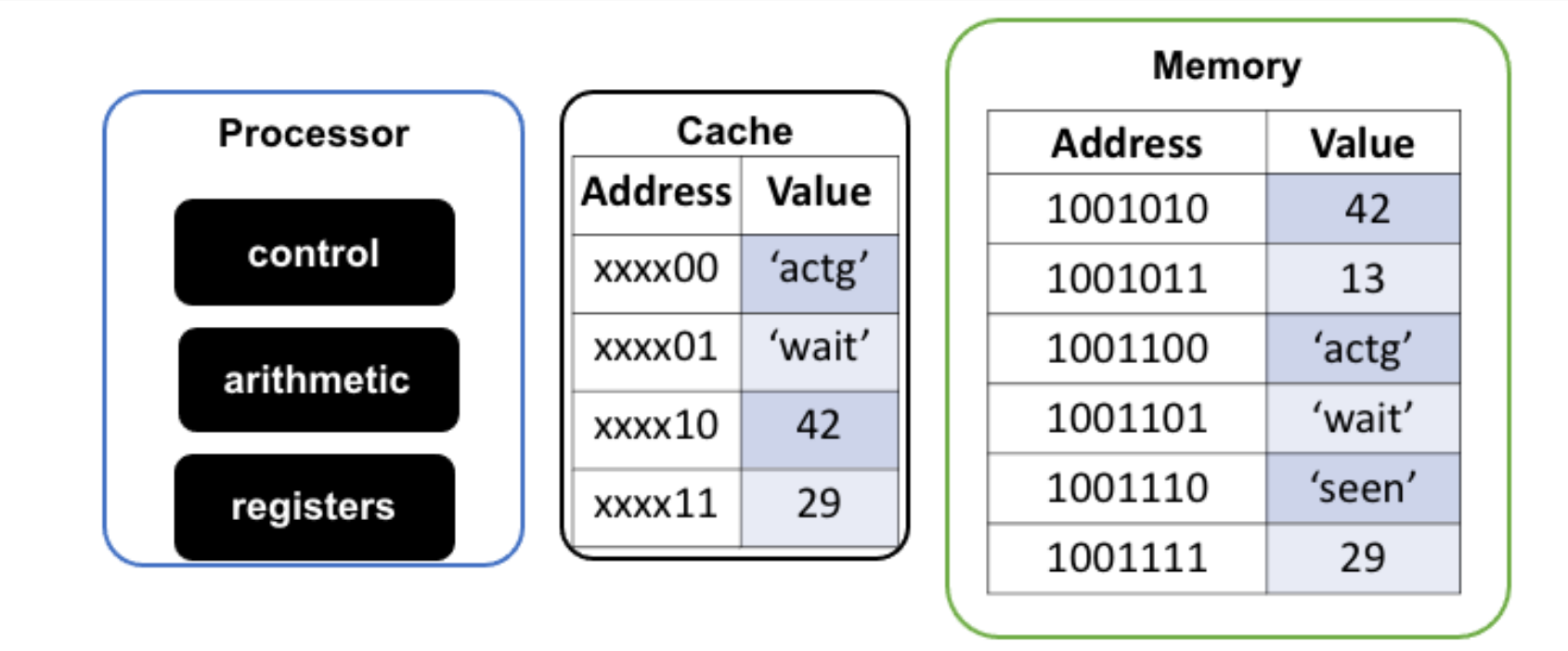
Programs usually have *locality*

- *Spatial locality*: things close to each other tend to be accessed consecutively
- *Temporal locality*: use a “working set” of data repeatedly

Cache hierarchy built to use locality.

Cache basics

- Cache is fast (expensive) memory which keeps copy of data; it is hidden from software
  - Cache hit: in-cache memory access—cheap
  - Cache miss: non-cached memory access—expensive
    - Need to access next, slower level of memory



Ex: data at memory address xxxx11 is stored at cache location 11

Cache basics

- Cache line length: # of bytes loaded together in one entry
  - Ex: If either xxxxx1100 or xxxxx1101 is loaded, both are

Cache	
Address	**Value**
xx1100	'actg'
xx1101	'wait'
xx1110	42
xx1111	29

- Full associative – a new line can be placed at any location in the cache.
- Direct mapped – each cache line has a unique location in the cache to which it will be assigned.
- n-way set associative – each cache line can be placed in one of n different locations in the cache.

Memory Hierarchy

- Most programs have a high degree of locality in their accesses
  - spatial locality: accessing things nearby previous accesses (makes use of multiple elements transferred together)
  - temporal locality: reusing an item that was previously accessed (makes use of efficient hierarchical accesses)
- Memory hierarchy tries to exploit locality to improve access

```
1  double z[100];
2  ...
3  sum = 0.;
4  for (i = 0; i < 1000; i++)
5     sum += z[i];
```

Approaches to Handling Memory Latency

- Eliminate memory operations by saving values in small, fast memory (cache) and reusing them
  - need temporal locality in program
- Take advantage of better bandwidth by getting a chunk of memory and saving it in small fast memory (cache) and using whole chunk
  - need spatial locality in program
- Take advantage of better bandwidth by allowing processor to issue multiple reads to the memory system at once
  - concurrency in the instruction stream, e.g. load whole array, as in vector processors; or prefetching
  - vector operations require access set of locations (typically neighboring), requires they are independent
- Overlap computation and memory operations
  - issue multiple reads/writes in parallel
  - delayed writes (write buffering) stages writes for later operation (require nothing dependent is happening)

Concurrency

- Take advantage of better bandwidth by allowing processor to issue multiple reads to the memory system at once
  - concurrency in the instruction stream, e.g. load whole array, as in vector processors; or prefetching
  - vector operations require access set of locations (typically neighboring), requires they are independent
- Overlap computation and memory operations
  - issue multiple reads/writes in parallel
  - delayed writes (write buffering) stages writes for later operation (require nothing dependent is happening)

How much concurrency do you need to run at bandwidth speed rather than latency:
concurrency = latency * bandwidth (Example: 10 sec latency* 2 Bytes/sec bandwidth requires 20 bytes concurrency running, i.e. 20 independent things to issue)

Memory Hierarchy

```
int sum1(int k, int a[])
{
int i, tmp =0;
for(i = 0; i < k; i++)
  tmp += a[i];
return tmp;
}
```

```
int sum2(int k, int *a)
{
int i, tmp =0;
for(i = 0; i < k; i+=4)
  tmp += a[i]+a[i+1]+a[i+2]+a[i+3];
return tmp;
}
```

```
int sum3(int k, int *a)
{
int i, tmp =0;
for(i = 0; i < k; i+=4)
  tmp += *a+*(a+1)+*(a+2)+*(a+3);
return tmp;
}
```

Teaser

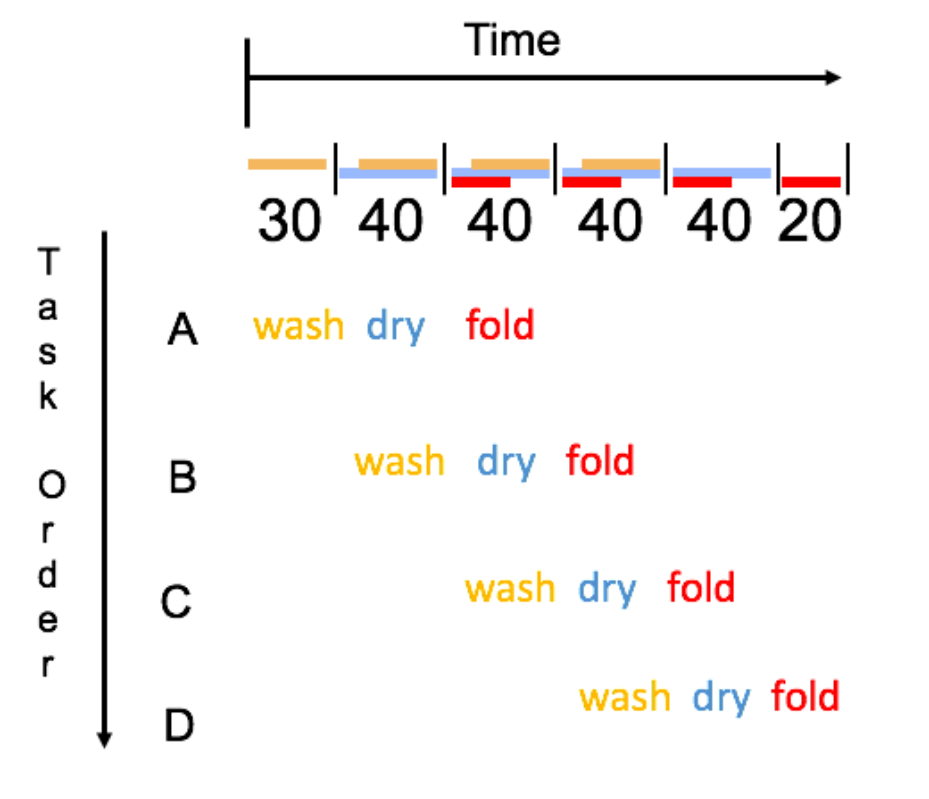
We have $N = 10^6$ two-dimensional coordinates, and want their centroid. Which of these is faster and why?

1. Store an array of (x_i, y_i) coordinates. Loop i and simultaneously sum the x_i and the y_i.
2. Store an array of (x_i, y_i) coordinates. Loop i and sum the x_i, then sum the y_i in a separate loop.
3. Store the x_i in one array, the y_i in a second array. Sum the x_i, then sum the y_i.

Pipelining

Latency: wash (30 min) + dry (40 min) + fold (20 min) = 90 min

- Sequential execution takes 4 * 90min = 6 hours
- Pipelined execution takes 30+4*40+20 = 3.5 hours
- Bandwidth = loads/hour = 4/6 l/h w/o pipelining = 4/3.5 l/h w pipelining $\leq$ 1.5 l/h w pipelining
  - Pipelining doesn't change latency (90 min)
  - Bandwidth limited by slowest pipeline stage
  - Speedup $\leq$ # of stages



Laundry pipelining example: 4 people doing laundry

Implicit Parallelism: Pipelining

Parallelism can be introduced at instruction level:

- The basic instruction cycle is broken up into a series of instructions.
- Example C = A + B

Assembly instructions:
load R1, @A
load R2, @B
add R1, R2
store R1, @C

Sequential vs Pipilining

Consider a simple serial code:

```
1  double x[100], y[100], z[100];
2  for (i = 0; i < 100; i++)
3    z[i] = x[i] + y[i];
```

Sequential Operation:

Fetch→Add→Normalize→Store

Fetch→Add→Normalize→Store

Pipelining:

Fetch→Add→Normalize→Store

Fetch→Add→Normalize→Store

Fetch→Add→Normalize→Store

Fetch→Add→Normalize→Store

Another improvement: Vector processor pipeline.

Pipelining example

Add 9.8e4 and 6.54e3

Time	Operation	Operand 1	Operand 2	Result
1	Fetch operands	9.87×10^4	6.54×10^3	
2	Compare exponents	9.87×10^4	6.54×10^3	
3	Shift one operand	9.87×10^4	0.654×10^4	
4	Add	9.87×10^4	0.654×10^4	10.524×10^4
5	Normalize result	9.87×10^4	0.654×10^4	1.0524×10^5
6	Round result	9.87×10^4	0.654×10^4	1.05×10^5
7	Store result	9.87×10^4	0.654×10^4	1.05×10^5

- Assume each operation takes one nanosecond.
- This takes about 7 nanoseconds.

Sequential vs Pipilining

Consider a simple serial code:

```
1  double x[100], y[100], z[100];
2  for (i = 0; i < 100; i++)
3    z[i] = x[i] + y[i];
```

- One floating point operation still takes 7 nanosecond.
- Divide the floating point adder into 7 separate pieces of hardware or functional units.
- First unit fetches two operands, second unit compares exponents, etc.
- Output of one functional unit is input to the next.
- With pipelining, 100 floating point additions now take 106 nanoseconds.

What have we learned?

- Details of machine are important for performance
- Processor and memory system
- Understanding of hardware limits
- Machines have memory hierarchies
- Caches are fast (small) memory that optimize average case
- To read from main memory is slow
- Before you parallelize, make sure you're getting good serial performance
- Pipelining
- Goal = minimize communication = data movement